GUIA VISUAL 068

CUT · SORT · UNIQ · TR
A CAIXA DE TEXTO

*O relatório dizia que o diretório mais pesado era o `/var/cache`, com 900 MB — mas o `/var/lib` tinha 1,5 GB e nem apareceu. O `du -sh | sort -rn` comparou os **números** 1 e 900: o `-n` só entende dígitos e **ignora a unidade**. No mesmo relatório, a contagem de endereços únicos veio baixa demais: o `uniq` compara **apenas linhas adjacentes**. Dois erros silenciosos, a mesma raiz — **supor que a ferramenta faz o que o nome sugere**.*

068
GUIAS LINUX

O PIPELINE CANÔNICO DE CONTAGEM — ESTÁGIO POR ESTÁGIO

`acesso.log`	→ `awk '{print $1}'`	→ `sort`	→ `uniq -c`	→ `sort -rn`
10.0.0.1 … 200	10.0.0.1	10.0.0.1	1 10.0.0.1	3 10.0.0.2
10.0.0.2 … 404	10.0.0.2	10.0.0.2	3 10.0.0.2	2 10.0.0.3
10.0.0.3 … 200	10.0.0.3	10.0.0.2	2 10.0.0.3	1 10.0.0.1
10.0.0.2 … 500	10.0.0.2	10.0.0.2		
10.0.0.3 … 200	10.0.0.3	10.0.0.3	colapsa e conta os grupos	ordena pelo número, do maior
10.0.0.2 … 200	10.0.0.2	10.0.0.3		
	extrai a coluna que interessa	**agrupa** as iguais lado a lado		

Tire o `sort` do meio e o resultado fica errado sem avisar. Sobre a entrada em ordem cronológica, o `uniq -c` veria **seis grupos de um** — `1 10.0.0.1`, `1 10.0.0.2`, `1 10.0.0.3`, `1 10.0.0.2` … — em vez de três endereços com 3, 2 e 1 ocorrências. Nenhum erro, nenhum aviso, só um relatório errado.

O `sort -rn` final funciona porque o `uniq -c` põe a contagem **na frente**, alinhada com espaços à esquerda — e o `-n` ignora esses espaços. Para desempatar pelo endereço, `sort -k1,1rn -k2,2`. **O mesmo resultado sai de um `awk` só** — `awk '{n[$1]++} END{for(k in n) print n[k], k}' | sort -rn` —, numa passada em vez de três, o que importa em arquivo grande. E para deduplicar **preservando a ordem de aparição**, o `sort` está fora: é `awk '!visto[$0]++'`.

A ARMADILHA DE CADA FERRAMENTA

`sort -k2n`

```
alfa  10 zzz
beta   9 aaa
```

"do campo 2 **até o fim da linha**". Compara

```
10 zzz
```

contra

```
9 aaa
```

como se fosse um número só — o resto da linha entra na conta. **Às vezes acerta, o que é pior que errar sempre.**

`sort -k2,2n`

```
alfa  10 zzz
beta   9 aaa
```

"do campo 2 **ao campo 2**". Compara

```
10
```

contra

```
9
```

, e só. É o que se queria dizer em praticamente todo uso de

```
-k
```

. A regra: **sempre escreva o campo final.**

O delimitador vem de `-t`; sem ele o `sort` corta na *transição* de espaço para não-espaço, e o campo inclui o espaço à esquerda — daí existir o modificador `b`. **Várias chaves se acumulam e valem em ordem**, cada uma com seus modificadores: `sort -t: -k3,3n -k1,1` ordena por UID e desempata por nome; em `-k2,2nr -k1,1`, o `r` vale só para a primeira.

SORT · AS OPÇÕES QUE DECIDEM O RESULTADO

`-h`	numérica **com sufixo humano** K, M, G — **é o conserto da abertura**
`-V`	versão: `1.9` antes de `1.10`
`-u · -s · -c`	remove duplicatas · estável (empate mantém a entrada) · só **verifica** se já está ordenado
`LC_ALL=C`	a localidade **define a ordem**: em `pt_BR` o `sort` ignora caixa e pontuação; sob `C` é a ordem dos bytes — previsível **e mais rápida**
`-S · -T`	tamanho do buffer e onde ficam os temporários — importa quando `/tmp` é pequeno ou é `tmpfs`

UNIQ · SÓ ENXERGA O VIZINHO

`-c · -d · -u`	conta · só as repetidas · só as que aparecem **uma única vez**
`-f2 · -w8`	ignora os 2 primeiros campos · compara só os 8 primeiros caracteres
`sort -u`	mais rápido que `sort | uniq`, porque elimina **durante** a ordenação — mas `-c`, `-d` e `-u` são do `uniq` e exigem o par completo

CUT · RÁPIDO, PREVISÍVEL E COM TRÊS LIMITES

`-d é 1 char`	não existe delimitador de dois espaços nem por expressão regular
`não colapsa`	espaço repetido desloca tudo — normalize antes com `tr -s ' '`
`não reordena`	`cut -f3,1` devolve **1,3**: o `cut` ignora a ordem pedida. Reordenar é `awk '{print $3, $1}'`
`-c1-8 · -s`	corta por **caractere** · suprime as linhas **sem** o delimitador (que por padrão saem inteiras)

TR · CARACTERE A CARACTERE, SÓ DO STDIN

`tr -d '\r'`	conserta arquivo do Windows — a causa do `bad interpreter: /bin/bash^M`
`tr -s ' '`	**colapsa** espaços repetidos: é o que torna o `cut -d' '` viável
`tr -c … '\n'`	`tr -c '[:alnum:]' '\n' | sort | uniq -c` é o contador de palavras de uma linha
`não é regex`	não conhece padrão e **não substitui palavras** — e `-d` com `-s` juntos raramente faz o esperado

O RESTO DA CAIXA

`comm -13 -23 -12`	**o que mudou entre duas listas**: só no 2º · só no 1º · em ambos
`paste -sd,`	lado a lado · `-s` junta **todas** as linhas numa só
`join -t: -1 1`	junção relacional — **exige as duas entradas ordenadas** pelo campo, ou perde pares em silêncio
`column -t`	alinha para ler · `nl` numera · `tac` inverte a ordem · `rev` inverte os caracteres
`tail -n +2`	tudo **a partir** da linha 2 — o jeito de pular cabeçalho
`shuf -n 100`	amostra aleatória · `split -l` fatia arquivo grande
`tee copia.txt`	bifurca no meio do pipeline: **a ferramenta de depuração** de pipeline longo (`tee /dev/stderr` mostra na tela e segue)

`rev | cut -d/ -f2 | rev` pega o **penúltimo** campo de uma linha de tamanho variável sem `awk`. E `comando | head -5` **mata o comando anterior com `SIGPIPE`** — é o comportamento desejado, o que faz `zcat enorme.gz | head` ser instantâneo. Só que sob `set -o pipefail` isso aparece como **falha do pipeline**: é a causa mais comum de script que "falha" fazendo exatamente o certo.

`sort arquivo > arquivo` **esvazia o arquivo.** O shell **trunca o destino antes** de o `sort` ler qualquer coisa — o conteúdo se perde